

VELUM

Plataforma de Detección de Fraude en Tiempo Real

Manual de Arquitectura, Algoritmos de Ventana Deslizante (Sliding Window), Integridad Criptográfica y Manual de Comandos Operativos

Componente	Especificación Técnica
Lenguaje / Runtime	Python 3.14 / Asynchronous WSGI con Uvicorn
Framework Web / API	FastAPI con Pydantic v2 (Esquemas de Entrada y Salida)
Capa ORM / Persistencia	SQLAlchemy 2.0 (Strict Typing) / Alembic Migrations
Bases de Datos Soportadas	SQLite (Desarrollo con modo WAL) / PostgreSQL 15+ (Producción)
Motor de Detección	Sliding Window en memoria (Deque atómicos + Threading Locks)
Integridad Transaccional	SHA-256 Canónico con hmac.compare_digest (Resistente a Timing Attacks)
Frontend / Dashboard	Vanilla JS + CSS moderno + Chart.js local (Sin CDNs externos)
Fecha de Compilación	01 de October de 2026

ESTADO DE CERTIFICACIÓN Y VALIDACIÓN DEL SISTEMA

- Pruebas Unitarias y de Integración (pytest):** 29 pruebas ejecutadas y aprobadas (100% éxito).
- Casos Funcionales en Vivo (verify_cases.py):** C1 a C6 aprobados exhaustivamente.
- Idempotencia y Resiliencia Concurrente:** Validado con pruebas de estrés de múltiples hilos en paralelo.
- Autotest de Criptografía WebCrypto:** Concordancia exacta de vectores SHA-256 entre frontend y backend.

1. Resumen Ejecutivo y Propósito del Sistema

VELUM es un motor de detección y prevención de fraude financiero diseñado para procesar transacciones electrónicas en tiempo real con una latencia inferior a 50 milisegundos. Su propósito fundamental es identificar y mitigar patrones de fraude de alta velocidad (ráfagas de transacciones, ataques de fuerza bruta sobre pasarelas de pago y operaciones automatizadas ilegítimas) antes de que causen pérdidas financieras irreparables.

A diferencia de los sistemas tradicionales basados exclusivamente en procesamiento por lotes (batch) o consultas pesadas a bases de datos relacionales, VELUM implementa una arquitectura híbrida de **Ventana Deslizante (Sliding Window) en memoria** respaldada por persistencia transaccional ACID. Cada evento transaccional es inspeccionado en el momento exacto de su arribo, asegurando una evaluación inmediata, atómica y segura.

Pilares Clave del Diseño

- 1. Garantía de Integridad Transaccional:** Cada solicitud entrante contiene una firma hash generada por el cliente. El servidor recalcula el hash mediante un formato canónico estricto y efectúa una comparación inmune a ataques de temporización (timing attacks). Si los datos son alterados en tránsito, la solicitud se rechaza inmediatamente con código HTTP 400 sin persistencia.
- 2. Detección Temporal por Ventana Deslizante:** Las ráfagas se monitorean dentro de una ventana móvil de 3 segundos exactos basada en la hora de recepción del servidor (received_at), impidiendo manipulaciones del reloj del cliente.
- 3. Cálculo Dinámico de Severidad por Franjas Horarias:** El nivel de riesgo (BAJO, MEDIO, ALTO, CRÍTICO) no es estático; se adapta al ciclo operativo comercial de Colombia (hora de Bogotá), evaluando el volumen contra umbrales de referencia diurnos, vespertinos y nocturnos.
- 4. Idempotencia Rigurosa y Manejo de Conflictos:** Permite a los clientes reintentar solicitudes de forma segura. Si una transacción idéntica ya fue procesada, responde HTTP 200 sin duplicar cobros ni ensuciar la ventana. Si se reutiliza un identificador con datos distintos, devuelve HTTP 409 Conflict.
- 5. Protección Auditada de Usuarios Bloqueados:** Intentos transaccionales de cuentas bloqueadas son registrados en base de datos bajo el estado RECHAZADA para fines forenses y retornan inmediatamente HTTP 403 Forbidden.

2. Arquitectura General y Capas del Software

VELUM está estructurado siguiendo los principios de arquitectura limpia y separación de responsabilidades en capas desacopladas:

Capa	Módulos Principales	Función y Responsabilidad
Presentación e Ingesta	app/routers/transactions.py app/routers/dashboard.py app/routers/simulator.py	Exposición de endpoints REST, serialización/deserialización Pydantic, manejo uniforme de respuestas y excepciones HTTP.
Seguridad & Integridad	app/services/hash_service.py app/security.py	Construcción de cadena canónica, cálculo SHA-256, comparación en tiempo constante (hmac.compare_digest) y validación de clock skew.
Motor de Detección	app/detector.py	SlidingWindowDetector singleton, deque en memoria por usuario, purga de eventos > 3s, cálculo de severidad por franjas.

Orquestación del Negocio	app/services/transaction_service.py app/services/dashboard_service.py	Gestión del ciclo de vida transaccional, resolución de usuarios, serialización de escrituras con <code>_db_write_lock</code> , consolidación de anomalías.
Persistencia y Datos	app/models.py app/database.py migrations/	Modelos SQLAlchemy declarativos (Usuario, Transaccion, Anomalia), SessionLocal, llaves foráneas e índices únicos. Migraciones Alembic.
Frontend & Monitoreo	templates/index.html static/js/*.js static/vendor/chart.umd.js	Dashboard interactivo en tiempo real con 5 KPIs, 4 gráficos analíticos, visualizador cronológico de ventana y simulador de ataques WebCrypto.

Estrategia de Concurrencia y Sincronización

Para evitar condiciones de carrera (race conditions) y lecturas fantasmas en entornos de alto tráfico, VELUM implementa un esquema de doble barrera:

- 1. Locks de Nivel de Usuario en Memoria:** En `SlidingWindowDetector`, cada dirección de correo dispone de un `threading.Lock` dedicado. Esto garantiza que dos solicitudes simultáneas del mismo usuario no corrompan el deque de timestamps, mientras que las transacciones de usuarios distintos se procesan en paralelo sin bloqueo mutuo.
- 2. Bloqueo Transaccional de Escritura (`_db_write_lock`):** Para motores de base de datos como SQLite (que utilizan bloqueo a nivel de archivo para escrituras), la función `process_transaction` sincroniza la verificación de idempotencia y la inserción de registros mediante un semáforo global, impidiendo colisiones de sesión y errores 'database is locked'.

3. Motor de Detección 'Sliding Window' y Severidad

El corazón de la detección de anomalías radica en el algoritmo de ventana deslizante temporal. El sistema no mide bloques fijos de tiempo (como horas o minutos calendario), sino un intervalo relativo móvil definido por la expresión: $[t_{repcion} - 3s, t_{repcion}]$.

Mecánica Paso a Paso de la Ventana

- **1. Captura del Tiempo del Servidor:** Al arribar la petición HTTP, el servidor asigna 'received_at = datetime.now(UTC)'. Se rechaza cualquier suposición de que el cliente tiene el reloj sincronizado.
- **2. Obtención de Lock del Usuario:** Se adquiere el Lock exclusivo del usuario. Ninguna otra hebra puede mutar la ventana de este usuario durante esta fase.
- **3. Purga de Eventos Vencidos:** Se evalúa el extremo izquierdo del deque. Cualquier timestamp con fecha estrictamente anterior a 'received_at - 3s' es expulsado (popleft).
- **4. Inserción del Nuevo Evento:** Se anexa el timestamp actual al deque. La longitud del deque representa exactamente el número de transacciones en la ventana de 3 segundos.
- **5. Comparación con Umbral Base:** Si 'count >= 3' (BASE_TRANSACTION_THRESHOLD), se dispara la bandera 'anomaly_detected = True'. La transacción se clasifica como SOSPECHOSA; de lo contrario, se clasifica como APROBADA.

Cálculo Dinámico de Severidad y Franjas Horarias

Un volumen de 3 transacciones en 3 segundos puede ser normal en un pico comercial matutino, pero altamente anómalo a las 3:00 AM. Por ello, la severidad se calcula contrastando el conteo de la ráfaga con la referencia esperada para la franja horaria en **hora oficial de Bogotá (UTC-5)**:

Franja Horaria	Intervalo Horario (Bogotá)	Referencia (Ref)	Naturaleza Operativa
Mañana	[05:00, 12:00)	10	Alto volumen transaccional comercial y laboral.
Tarde	[12:00, 20:00)	6	Volumen regular de pagos, consumo y compras.
Noche	[20:00, 05:00)	3	Bajo volumen; umbral de alerta muy sensible a ráfagas.

Fórmula de Razón (Ratio) y Niveles de Alerta:

Rango de Ratio = Conteo / Ref	Nivel Asignado	Color en Dashboard	Acción Recomendada
Ratio < 0.5	BAJO	Verde / Azul	Registro y monitoreo pasivo de telemetría.
0.5 ≤ Ratio < 1.0	MEDIO	Amarillo ámbar	Alerta en panel de monitoreo operativo.
1.0 ≤ Ratio < 2.0	ALTO	Naranja oscuro	Revisión prioritaria por analista antifraude.
Ratio ≥ 2.0	CRITICO	Rojo carmesí	Escalamiento urgente y bloqueo temporal preventivo.

Consolidación de Ráfagas (Una Anomalía por Incidente)

Si un usuario envía 10 transacciones en 2 segundos, generar 8 registros de anomalía independientes saturaría la mesa de operaciones. VELUM implementa **consolidación de anomalías solapadas**: si ya existe una anomalía con estado NUEVA o ABIERTA para ese usuario dentro de la ventana de 3 segundos, el sistema actualiza la fila existente (incrementando cantidad_transacciones, actualizando el nivel a la severidad más alta y estableciendo estado_revision = 'ABIERTA') en lugar de crear filas redundantes.

4. Protocolo Criptográfico y Formato Canónico

Para certificar que una transacción no fue alterada por intermediarios, atacantes man-in-the-middle (MITM) o fallas en el canal de comunicación, VELUM exige la inclusión de un hash SHA-256 generado sobre una cadena normalizada canónica.

Reglas Estrictas de Construcción de la Cadena Canónica:

La cadena canónica sigue el esquema: idTxn|user|date|value|paymentMethod.

Campo	Transformación Canónica Exigida	Ejemplo Válido
idTxn	Cadena única alfanumérica (1 a 64 caracteres). Sin alteración.	TXN-2026-001
user	Email en minúsculas estrictas y sin espacios en blanco intermedios ni laterales.	juan.perez@empresa.com
date	Formato ISO 8601 con exactamente 3 dígitos de milisegundos: YYYY-MM-DDTHH:MM:SS.mmm	2026-09-23T10:30:01.120
value	Monto formateado con exactamente dos cifras decimales separadas por punto.	150000.00
paymentMethod	Uno de los 4 valores permitidos: Tarjeta, PSE, Transferencia, Otro.	Tarjeta

Ejemplo de Generación:

Cadena Canónica:
TXN-7788|carlos@banco.co|2026-10-01T14:22:05.500|99500.00|PSE

Hash SHA-256 Resultante (64 caracteres hex minúsculas):
7d12f3bc8a4b09e1e21b033d59e35b7194f1c1f5106198f7975b94f1d6928e44

Mecanismos de Resistencia a Ataques:

- **Mitigación de Timing Attacks:** Las cadenas hash nunca se comparan usando operadores convencionales de igualdad ('=='), los cuales retornan 'False' en el primer byte discordante, permitiendo al atacante deducir caracteres mediante análisis de tiempos. VELUM emplea estrictamente hmac.compare_digest(hash_esperado, hash_recibido), ejecutando la validación en tiempo constante.
- **Validación de Desviación de Reloj (Clock Skew):** El campo 'date' del cliente se contrasta contra la hora UTC del servidor. Si la discrepancia absoluta supera MAX_CLOCK_SKEW_SECONDS (por defecto 300 segundos = 5 minutos), la transacción se aborta con error HTTP 422.
- **Política Cero Persistencia en Hash Inválido:** Las transacciones con firma corrupta o adulterada se descartan con HTTP 400 antes de interactuar con la base de datos o con la ventana deslizante, previniendo ataques de denegación de servicio (DoS) por llenado de almacenamiento.

5. Comandos Críticos y Manual de Operaciones

A continuación se presenta el compendio exhaustivo de comandos requeridos para la administración, puesta en marcha, verificación de calidad y mantenimiento del sistema:

Objetivo Operativo	Comando PowerShell / Terminal	Descripción Técnica
Iniciar Servidor (Dev)	<code>cd appresso venv\Scripts\uvicorn app.main:app --port 8000 --reload</code>	Inicia FastAPI con recarga en caliente en <code>http://localhost:8000</code> . Reconstruye ventanas activas al arrancar.
Ejecutar Pruebas (pytest)	<code>venv\Scripts\pytest -v</code>	Corre las 29 pruebas automatizadas (unitarias, concurrencia de hilos, integridad hash, endpoints).
Validación en Vivo (C1-C6)	<code>venv\Scripts\python verify_cases.py</code>	Ejecuta la suite funcional de extremo a extremo contra el servidor HTTP activo para certificar los casos C1 a C6.
Poblar Base de Datos (Seed)	<code>venv\Scripts\python seed.py</code>	Inserta 8 usuarios de prueba y 52 transacciones sintéticas con todas las severidades y métodos de pago.
Aplicar Migraciones BD	<code>venv\Scripts\alembic upgrade head</code>	Aplica las migraciones pendientes en SQLite/PostgreSQL sincronizando el esquema con <code>models.py</code> .
Crear Nueva Migración	<code>venv\Scripts\alembic revision --autogenerate -m "descripcion"</code>	Inspecciona cambios en modelos SQLAlchemy y genera el script de migración correspondiente.
Despliegue con Docker	<code>docker compose up --build -d</code>	Levanta los contenedores orquestados de VELUM (FastAPI) y PostgreSQL con volumen de datos persistente.
Detener Contenedores	<code>docker compose down</code>	Detiene y remueve los contenedores de la aplicación y la base de datos de manera limpia.
Inspeccionar Logs en Vivo	<code>docker compose logs -f app</code>	Transmite en tiempo real los registros de auditoría y excepciones del contenedor de la aplicación.

Protocolo de Verificación Funcional (Casos C1 al C6):

El script `verify_cases.py` evalúa los 6 escenarios de aceptación fundamentales del sistema:

- **C1 — Transacción Única Normal:** Envía una transacción aislada. Debe aprobarse (201 Created, `status='APROBADA'`, `anomalyDetected=False`).
- **C2 — Transacciones Espaciadas:** Envía una segunda transacción tras 3.5 segundos (>3s). Debe aprobarse sin disparar alertas de ráfaga.
- **C3 — Detección de Ráfaga de Fraude:** Envía 3 transacciones en menos de 1 segundo para el mismo usuario. La 3ra debe marcarse SOSPECHOSA con `anomalyDetected=True`.
- **C4 — Aislamiento entre Usuarios:** Envía 3 transacciones para el Usuario A y 1 para el Usuario B. Verifica que el Usuario B no sea penalizado por el tráfico de A.
- **C5 — Intento de Manipulación de Hash:** Envía una transacción con el monto adulterado en el payload. El servidor debe responder 400 Bad Request sin persistir en BD.
- **C6 — Idempotencia de Duplicados:** Reenvía una transacción previamente aprobada. El servidor debe responder 200 OK con `duplicate=True` sin crear registros adicionales.

6. Catálogo de Endpoints de la API REST

1. POST /api/transacciones — Ingesta de Transacción

Endpoint principal para procesar, auditar y detectar anomalías en transacciones financieras.

Código HTTP	Condición / Escenario	Persistencia en BD	Estructura de Respuesta
201 Created	Transacción nueva legítima (APROBADA o SOSPECHOSA si supera umbral).	Sí (Transaccion + Anomalia si aplica)	{"success": true, "transaction": {...}, "analysis": {...}}
200 OK	Duplicado idéntico (mismo idTxn y mismo hash SHA-256). Idempotente.	No (reutiliza existente)	{"success": true, "duplicate": true, ...}
400 Bad Request	Firma hash no coincide con la recalculada en el servidor.	No (Cero persistencia)	{"success": false, "error": {"code": "HASH_INVALID"}}
403 Forbidden	Usuario remitente se encuentra en estado BLOQUEADO o INACTIVO.	Sí (Se guarda como RECHAZADA)	{"success": false, "error": {"code": "USER_BLOQUEADO"}}
409 Conflict	idTxn existente pero con contenido o monto diferente (colisión/fraude).	No	{"success": false, "error": {"code": "CONFLICT"}}
422 Unprocessable	Fallo de validación Pydantic (fechas sin ms, montos negativos) o clock skew.	No	{"success": false, "error": {"code": "VALIDATION_ERROR"}}
500 Internal Error	Error no controlado. No expone trazas de pila (stack traces) al cliente.	Rollback transaccional	{"success": false, "error": {"code": "INTERNAL_ERROR"}}

2. GET /api/dashboard/stats — Métricas y Agregaciones

Calcula métricas clave para periodos: ?periodo=hoy, semana o mes.

- **KPIs agregados:** Total transacciones, Aprobadas, Sospechosas, Rechazadas, Monto bajo riesgo.
- **Detección de Hora Pico:** Identifica la hora del día cuya cantidad de anomalías supera: **Media + (2 × Desviación Estándar)**.
- **Cálculo de Tendencia:** Variación porcentual de volumen frente al periodo anterior equivalente.
- **Usuarios Recurrentes:** Cantidad de clientes con 2 o más anomalías registradas en la ventana.

3. GET /api/dashboard/timeline-sliding-window/{usuario_id}

Reconstruye retrospectivamente la historia transaccional del usuario seleccionado, computando para cada evento si estuvo dentro de la ventana de 3 segundos, los límites exactos [inicio, fin], el conteo instantáneo y la severidad asociada para alimentar el visualizador gráfico interactivo.

7. Frontend, Simulador y Buenas Prácticas

Arquitectura del Panel de Control (Dashboard)

El panel de monitoreo se implementó sin dependencias complejas (sin React, Angular ni Vue), utilizando JavaScript moderno vanilla estructurado en módulos orientados a eventos:

- **dashboard.js:** Controlador principal de telemetría. Consulta periódicamente `/api/dashboard/stats` y actualiza los 5 KPIs de cabecera y los 4 gráficos interactivos renderizados mediante Chart.js 4.4.4 (alojado localmente en `static/vendor/chart.umd.js`, garantizando funcionamiento autónomo sin acceso a Internet).
- **sliding-window.js:** Visualizador de ventana móvil. Presenta una línea temporal interactiva con estados codificados por colores y detalles de solapamiento temporal.
- **simulator.js:** Motor de simulación en el navegador. Utiliza la API nativa `window.crypto.subtle.digest` (WebCrypto) para calcular firmas SHA-256 en tiempo real directamente en el cliente.

Botones del Simulador de Tráfico:

Acción en UI	Usuario Utilizado	Comportamiento y Resultado Esperado
■ Simular Ataque en Ráfaga	b@b.com	Despacha 3 transacciones en menos de 1.5 segundos. La 3ra dispara la alerta roja 'ANOMALÍA DETECTADA — POSIBLE_FRAUDE' con actualización inmediata de KPIs.
■ Simular Tráfico Normal	c@c.com	Envía 2 transacciones espaciadas por una pausa de 4.2 segundos. Ambas se aprueban con banner verde 'TRÁFICO NORMAL'.
■ Autotest Criptográfico	Vectores Oficiales	Calcula el hash de los 5 vectores canónicos en JavaScript WebCrypto y los compara con los esperados en Python, validando 100% de paridad.
■ Restablecer Datos	General	Limpia la base de datos de simulación y resetea las ventanas en memoria para permitir nuevas rondas de prueba.

Evolución a Producción y Escalabilidad Horizontal

Para entornos de producción con cientos de miles de transacciones por segundo distribuidas en clústeres:

1. **Sustitución de Memoria Local por Redis:** Reemplazar los deque de Python por estructuras Sorted Sets (ZSET) en Redis, donde el score sea el timestamp epoch en milisegundos. Esto permite que múltiples instancias de FastAPI compartan la misma ventana deslizante sin discrepancias.
2. **Autenticidad mediante HMAC:** Transicionar de SHA-256 plano a HMAC-SHA256 con llave secreta compartida por cliente para evitar que un atacante que conozca la fórmula matemática canónica falsifique firmas legítimas.
3. **Particionamiento de Base de Datos:** Aplicar particionamiento por rangos de fecha mensual en la tabla transacciones de PostgreSQL para optimizar consultas analíticas históricas.

Certificación de Entrega Técnica: Sistema auditado, verificado y documentado para despliegue.

Generado automáticamente por Antigravity Engineering | 2026-10-01 23:13:36